
DNG Compare
SAME PHOTOGRAPH. DIFFERENT STORIES.
Metadata A
Make NIKON
Model Z 8
DateTimeOriginal 2024:10:14 17:36:21
ISO 64
FocalLength 24.0 mm
ColorSpace Adobe RGB
Rating ★★★★☆
Keywords Sierra, river, sunset
Software NX Studio
GPSLatitude 37.8653 N
GPSLongitude 119.5384 W
DNG
IPTC
EXIF
XMP
TIFF
MakerNotes
Keywords
Ratings
Color Profile
Location
History
XMP
Metadata B
Make NIKON
Model Z 8
DateTimeOriginal 2024:10:14 17:42:03
ISO 64
FocalLength 24.0 mm
ColorSpace sRGB
Rating ★★★★☆
Keywords Sierra, California, river
Software Adobe Camera Raw
GPSLatitude 37.8653 N
GPSLongitude 119.5384 W
IMAGE PIXELS PRESERVED
SAME PIXELS RICHER STORIES
METADATA MAKES CONNECTIONS

DNG Compare

A manual for raw samples, metadata and exact recovery

Alexy Khrabrov, with Claude as co-author

2026-11-10

Version 2.1.0-eedd4fb7
Source commit eedd4fb7f0cd21f1dee05afb94ca5077585aad90

First Pair Press
firstpair.press

Contents

About this manual . . . 3
What DNG Compare is . . . 3
- Choose the question before the tool . . . 4
- Where it fits . . . 4

Part I: dngc . . . 5
- One command . . . 5
- Building . . . 5
- `dngc run` . . . 6
- `dngc restore` and `dngc versions` . . . 8
- `dngc verify` and `dngc status` . . . 8
- `dngc query` . . . 8
- Listing and pruning: `ls`, `drop`, `trash`, `keep` . . . 9

Part II: The archive . . . 10
- Layout . . . 10
- The byte-section rule . . . 10
- What a record holds . . . 10
- The metadata section . . . 11
- Kinds of archive . . . 11
- Migrating an older archive . . . 12
- Shuttles: wiped and refilled sources . . . 12
- Reading leaves the archive as it was . . . 13

Part III: Why preservation is the default . . . 13
Part IV: Implementations and performance . . . 14
- Three implementations, one archive . . . 14
- One read per pair . . . 14
- Measured on one archive . . . 14

Part V: The underlying tools . . . 15
- Install and build . . . 15
- A reproducible synthetic example . . . 16
- Python comparison command . . . 17
- How the Python image comparison works . . . 18
- Metadata extraction and categories . . . 19
- Interpret the Python reports . . . 20
- Resume and protect a comparison . . . 20
- Rust bounded scanner . . . 21
- Rust metadata reporter . . . 21
- Archive embedded XMP bytes . . . 22
- Preserve exact original files . . . 22
- Native byte workers . . . 23
- Full metadata snapshots and queries . . . 24
- Develop settings across editors . . . 24
- Rust and OCaml extraction pools . . . 26
- Year-scoped preservation workflow . . . 28
- Finalization and image categorization . . . 29
- SQLite and optional Turso . . . 29
- Measure workers without disturbing a real job . . . 30

Recovery after a failure 30
Boundaries of this release 31
Part VI: The archive format 31
What the format promises 31
Terms 32
Sections 32
The archive's own record 32
The byte section's database 32
Records and their three kinds 33
Restoring a content 33
A reference is a content, not a path 33
The comparison and metadata sections 34
Dropping, the trash and tombstones 34
Locks and reading 34
Conformance 35
Part VII: Skeletons 35
The problem: a reference that is still in use 35
What an edit actually changes 35
Blocks and the skeleton 36
Rebuilding a file from its edited version 36
The skeleton section 37
Writing skeletons 37
Restoring through a skeleton 37
What skeletons do not cover 38
Measured 38

About this manual

This manual is by Alexy Khrabrov, with Claude as co-author. The ideas and decisions are Alexy's; Claude wrote the text and the code with him.

This is the second edition. The first described a set of tools, each with its own command and output directory. Since then they have been joined under one command, `dngc`, writing one kind of archive, with the same behaviour in OCaml, OxCaml, Rust and Python. Parts I to IV describe that. Part V keeps the first edition's chapters on the underlying tools, which still work on their own and which `dngc` runs. Part VI is the archive format in full, for anyone who reads or writes an archive with their own code. Part VII is the skeleton method, which lets an archive survive edits to the reference copy it was preserved against.

The public source is firstpair/dng-compare. The README gives an overview. Every example below uses synthetic paths such as `/Volumes/Archive`; nothing here comes from a personal photo library except measured figures, which say where they were taken.

What DNG Compare is

A DNG can change without its sensor samples changing. Lightroom or another editor may write new XMP, update a preview, change a rating, or reorganize TIFF metadata. Whole-file checksums then differ, even when the photograph's stored raw samples are identical. Conversely, a matching file name, size or stored digest does not establish that two copies hold the same image.

DNG Compare separates those questions. It compares the actual image blocks of two copies, decodes when their encodings differ, reports metadata differences, and **preserves** one copy byte for byte as a verified delta against the other. The result is an archive from which every file of the source can be rebuilt exactly, as long as the reference copy is kept. The reference may go on being used: with skeletons (Part VII), a reference file that an editor has since written a rating or develop settings into still serves.

Choose the question before the tool

Identity	What is compared	Appropriate use
Whole-file identity	Every byte, including XMP, previews, padding and private data	Exact restoration, backup identity and a cloud original's whole-file checksum
Encoded image identity	Ordered image blocks and the selected decoding parameters	Confirm equal stored samples without decompression
Decoded sample identity	Complete arrays of stored sample values after decoding	Resolve different compression, strip layouts or byte order
Metadata equality	Typed fields exposed by ExifTool, under the stated exclusions	Explain settings, timestamps, camera data and layout differences
Rendered image equality	A processing pipeline's output after crop, profiles and edits	Not established by this project
Lightroom native fingerprint	Adobe's native original-digest calculation	Not generated or registered by this project

The distinction matters before any deletion. A source whose raw samples are represented elsewhere may still contain a unique develop history, embedded preview, original-file payload or private metadata. That is why DNG Compare preserves rather than only compares (Part III).

DNG extends TIFF and can carry raw data together with interpretation and other metadata. Adobe publishes the format specification and SDK. The comparator's selected tags and page classification are its implementation scope, not a claim of complete DNG conformance. Adobe's DNG resources.

Where it fits

This book is one of nine on the FirstPair photo tools. Each owns one part, and The FirstPair Photo Stack shows how they fit together.

- **tiffgraph** walks a TIFF or DNG file's structure under a fixed grammar and finds its editable XMP packet.
- **phidentify** computes a photo's identities: SHA-256, DNG raw blocks, image data, block marks, Lightroom fingerprint, metadata record and preserved digest, and the skeleton that rebuilds a file from an edited copy.
- **phindex** keeps those identities and a skeleton for every file of a volume, aligns two volumes by content, and restores a file as it was before a metadata edit.
- **DNG Compare** compares pairs and preserves one side byte for byte as deltas against the other, restorable even after the other side is edited, with a searchable metadata section, through `dngc`.

- **Semantic Unison** synchronizes photo trees, sending metadata edits without resending image bytes, keeps the versions it replaces as small skeletons, and preserves what a shuttle disk brings into the same archives.
- **packdir** keeps directories of small files as SQLite archives that still work like plain files.
- **Tursolite** is the SQLite and Turso layer under packdir, DNG Compare and Mechanic.
- **diskproxy** serves a folder as a virtual disk, such as previews with camera metadata spliced in, and captures ratings into sidecars.
- **Lightroom Repair** repairs Lightroom Classic catalogs, previews and Sync without forging their state.

DNG Compare owns two contracts the other books rely on: **the archive layout** (Part II), which Semantic Unison writes too, and **the engine rule**: metadata records made by Perl ExifTool and by exiftool-rs are separate identities and are never mixed in one archive.

Part I: dngc

One command

`dngc` puts comparison, preservation and metadata behind a few verbs that all work on one archive:

```
dngc run LEFT RIGHT ARCHIVE                  # the whole LEFT tree
dngc run LEFT RIGHT ARCHIVE --pairs P.tsv    # only the listed pairs
dngc status ARCHIVE                          # what the archive holds
dngc verify ARCHIVE --restore-sample N       # check; rebuild N at random
dngc restore ARCHIVE REL OUTPUT              # rebuild one file
dngc versions ARCHIVE [REL]                  # current and earlier contents
dngc restore ARCHIVE REL OUT --version L     # rebuild an earlier content
dngc restore ARCHIVE REL OUT --reference-root DIR   # the reference tree has moved
dngc restore ARCHIVE REL OUT --index I       # find a moved reference in a phindex index
dngc query ARCHIVE                           # most frequent changes
dngc query ARCHIVE files TAG                 # files where TAG changed
dngc query ARCHIVE values TAG                # with old and new values
dngc query ARCHIVE "SELECT ..."              # any SQL on the index
dngc index ARCHIVE                           # rebuild the metadata index
dngc ls ARCHIVE [--glob P] [--kind K] [-0]   # list records, for xargs
dngc drop ARCHIVE --glob P [--yes|--force]   # move records to the trash
dngc trash ARCHIVE [restore|empty|release]   # list or undo drops
dngc keep ARCHIVE --glob P [--yes]           # keep records as full copies
dngc migrate ARCHIVE [--dry-run]             # an older archive into bytes/
dngc skeleton build ARCHIVE [--workers N]    # let references be edited later
dngc skeleton info ARCHIVE                   # how many skeletons, by format
```

LEFT is the tree to preserve, the *source*. RIGHT is the tree it is preserved against, the *reference*. Nothing in either tree is ever written.

`dngc` is one native binary, built from the OCaml library that also builds `dng-archive`, under stock OCaml 5 and under OxCaml. A reference front end, `dngc.py`, runs the same verbs by calling the tools as programs; `./dngc` in the checkout picks the native binary when one is built. Both front ends give the same output, messages and exit statuses, and the tests run every case against both.

Building

```
make setup                                   # Python environment (uv)
make build-rust                              # Rust tools
```

```
cd ocaml-archive
opam exec --switch 5.4.0+ox -- \
  dune build --build-dir _build-5.4.0+ox bin/main.exe bin/dngc.exe
```

`bin/dngc.exe` is `dngc`, `bin/main.exe` is `dng-archive`. The same command with `--switch default` builds them with stock OCaml. The prerequisites (ExifTool, xdelta3, uv, Rust, opam) are in Part V, *Install and build*. The OCaml code uses Tursolite for its SQLite files, so the opam switch needs `firstpair-tursolite` pinned at the version in `ocaml-archive/dng-archive.opam`.

`dngc run`

Option	Values	Default	Meaning
`--pairs FILE`	`LEFT<TAB>RIGHT` lines	the whole tree	process only these pairs
`--bytes`	`delta`, `none`	`delta`	preserve each left file; `none` only compares
`--metadata`	`exiftool`, `phindex`, `none`	`exiftool`	compare metadata with Perl ExifTool, take it from phindex indexes, or skip it
`--index L R`	two phindex indexes	none	settle unchanged pairs without reading them; needed by `--metadata phindex`
`--impl`	`ocaml`, `rust`, `python`	`ocaml`	which implementation runs the pairs
`--workers N`		a quarter of the cores	pairs at a time, and ExifTool processes
`--max-mib R`	MiB/s	no limit	throttle reads, e.g. on one hard disk shared with other work
`--trust-files`		off	skip the final identity check; only when nothing changes either tree during the run

The whole tree

Without `--pairs`, every regular file under LEFT is covered. `.DS_Store` files and symbolic links are left out. Each file gets one of three kinds of record:

- **identical**: the reference holds the same bytes; the record keeps both SHA-256s and the file's attributes, and no payload;
- **delta**: the reference differs; the record keeps a VCDIFF delta that rebuilds the source from the reference, verified by decoding it again before it counts;
- **full copy**: no counterpart; the file itself is kept.

How a file finds its counterpart depends on `--index`. Without it, the same relative path is tried, then a file name that is unique under RIGHT. With phindex indexes of both trees, `phindex align` pairs files **by**

content, so moved, renamed and re-dated copies find their counterpart. LEFT must then be fully indexed, and `dngc` checks that before it starts. Files whose names begin with a dot, which align leaves out by design, are kept as full copies; some are unique, such as an interrupted copy's temporary file.

Phindex indexes

phindex keeps every file's identities in a SQLite index per volume. Given both indexes, `dngc` reads a file only when it has to:

- A row counts only when its size and modification time match the file now.
- A pair whose two rows have the same SHA-256 is identical. It is not read, and its record is written **in bulk**: in order, in one process, thousands to a transaction.
- `--metadata phindex` builds the archive's metadata section from the indexes' metadata records, so ExifTool reads nothing either. A file of a type phindex does not read metadata from, such as a Capture One `.cop` cache proxy, is listed in its row under `no_metadata` and counted as `files_without_metadata`, not as an error. A pair whose two sides have the same SHA-256 in current index rows has the same metadata by construction: its row keeps both stamps and `same_bytes_sha256` instead of the two records, which are neither read nor stored, and it counts as compared with no differences (`files_byte_identical`). It is left out of the per-tag coverage counts, which describe the records kept.
- The metadata step builds its rows in `--workers` forked processes and writes them in order; the batches are the same as with one worker. Its parent compacts its memory before it forks, so the workers start small.
- If either tree disappears during the metadata step, as a disk array that drops off does, the step stops before writing the next batch instead of recording an error for every remaining pair. Running the command again continues from there.

A repeated run

A second `dngc run` on the same archive reuses everything still valid: cached comparisons, preserved records and metadata batches. The metadata step learns which pairs it has recorded from the metadata index, which keeps each file's size, modification time and flags, so on 1.7 million recorded files it is ready in about 20 seconds; it parses only batches the index does not hold. Comparisons are cached by both files' inode, size, modification time and change time; the device number is left out, because macOS numbers a volume each time it is mounted. The pair list a run checked before is trusted, so the second run does not stat every listed file first.

Progress

Each step prints a `+` line naming the command it runs. The comparison prints a line per minute:

```
24,704/713,261 | raw equal=24,704 different=0 unverified=0 | 182.3s | cached=24,374 |
150.2 pairs/s, ETA 1h16m
```

The rate counts every pair, whether from the cache, settled from an index, or compared, over the last five lines, so the ETA follows the current mix of work. The preservation prints how many records the archive holds, and the bulk pass prints `identical files from the phindex indexes: N recorded of M new` every 30 seconds. The metadata step prints `reading recorded batches: …` while it reads back what an earlier run wrote, then `N/M new pairs` lines.

At the end, a run stats every file it preserved again, on both sides, and records an error for any whose identity changed during the run: an editor writing into a source, or a reference being replaced. On slow disks that takes minutes for a large run. `--trust-files` skips it, for runs on trees nothing else is using.

`run` exits 0 when every source file is preserved and every pair compared without error, 1 when something is incomplete or differs, 2 for a usage error, and 130 when interrupted. Running the same command again resumes.

`dngc restore` and `dngc versions`

```
dngc restore /Volumes/Archive/dngc/2024 photos/IMG_0001.DNG /tmp/IMG_0001.DNG
```

`restore` rebuilds one source file at a new path, checks its SHA-256 against the record, and restores its mode, times, flags and extended attributes. For a delta it reads the reference copy; if the reference tree has moved, `--reference-root DIR` says where it is now. If the reference file has been edited since and the archive has its skeleton, `restore` first rebuilds the reference as it was (Part VII).

If the reference file has been moved or renamed, give `restore` a phindex index of the tree it is in now:

```
dngc restore /Volumes/Archive/dngc/2024 photos/IMG_0001.DNG /tmp/IMG_0001.DNG \
  --index /Volumes/Backup/index/archive.sqlite
```

`--index FILE` may be repeated. `restore` asks each index which files hold the reference's SHA-256 and uses the first that still does. If the file was edited as well as moved, it asks which files share its image data and rebuilds the reference from one of them through its skeleton. The index is opened read-only and is never trusted: every file it names is checked by SHA-256 before it is used. `--candidates N` sets how many files are tried in each of those two steps (default 8; 0 turns the index off).

When the whole reference tree has moved and kept its layout, `--reference-root` is the direct way and needs no index. Both can be given: a reference is looked for below the new root first, then through the indexes. `dngc verify --restore-sample N` and `dngc keep` rebuild records too and take the same three options.

A path whose content changed between runs keeps every content it had. `versions` lists them, each under a label made of the source's modification time in UTC and the first 12 hex digits of its SHA-256, such as `20261029T171631Z-b755b147cbed`; `restore --version LABEL` rebuilds an earlier one.

`dngc verify` and `dngc status`

`verify` checks that the byte section has no errors, that every delta and full copy its records name exists, that the metadata index passes SQLite's integrity check, and that the comparison finished. It prints its progress on stderr. SQLite's check reads the index page by page, which on hard disks takes most of an hour for a few gigabytes, so an index over 256 MB is first copied, in order, to a local temporary file, and the copy is checked: minutes instead. `--restore-sample N` also rebuilds N records at random into a scratch directory, checking each SHA-256. Exit status 0 means no problems found. Run it before deleting anything a run preserved.

`status` prints, as JSON, what each section holds: files, errors, payload bytes, earlier versions, metadata files and the runs that wrote the archive.

`dngc query`

The metadata section is a SQLite index with one row per changed tag per file. `query` with no argument lists the most frequent changes; `files TAG` and `values TAG` list the files where a tag changed, with old and new values; any `SELECT` runs as it is. Part V, *Full metadata snapshots and queries*, describes the tables.

Listing and pruning: `ls`, `drop`, `trash`, `keep`

`ls` prints one record per line, the relative path by default, so a selection can feed other tools:

```
dngc ls ARCHIVE --glob '*.ini'                      # by pattern; --glob repeats
dngc ls ARCHIVE --kind delta                        # identical, delta or full
dngc ls ARCHIVE --changed Rating                    # metadata tag changed between the
trees
dngc ls ARCHIVE --glob '*.ini' --print reference -0 | xargs -0 ls -l    # the reference
copies
dngc ls ARCHIVE --kind full --json                  # one JSON object per record
```

Patterns are shell patterns on the whole relative path, so `*` crosses folders. `--print source` or `--print reference` prints those absolute paths instead, `-0` ends each entry with NUL for `xargs -0`, and `--trashed` lists the records in the trash.

`drop` removes records from the archive, in two stages. Selected by `--glob` or by a list (`--from FILE`, `-` for stdin, `-0` for NUL-separated), it first only lists them and says what it would move. With `--yes` it moves each record and its earlier versions into the trash: rows of table `trash` in the byte section, the payloads under `ARCHIVE/trash/DROP/`, and the metadata index rows into the index's own trash table. With `--force` it deletes them instead. Either way it leaves a **tombstone** per path, so a later `dngc run` does not preserve those files again, and says how many listed files it skipped. Every drop is recorded in `ARCHIVE.json` under `drops`.

```
dngc drop ARCHIVE --glob '*.ini'                # what it would drop
dngc drop ARCHIVE --glob '*.ini' --yes          # into the trash
dngc ls ARCHIVE --trashed --print reference -0 | xargs -0 trash   # their reference
copies, to the macOS Trash
```

`trash` lists the drops with their state (`trashed`, `restored`, `emptied`, `deleted`). `trash restore DROP` puts a drop back, records, earlier versions, payloads and metadata rows, and removes its tombstones; `trash empty [DROP] --yes` deletes what a drop moved to the trash, keeping its tombstones; `trash release DROP` removes a drop's tombstones, so later runs preserve those paths again. `restore` of a dropped path names the drop instead of reporting the path missing.

Mind the reference. An identical or delta record does not hold the file: restoring it needs the reference copy. Deleting reference files leaves such records unrestorable, and the archive cannot tell. Drop the records first, or keep them as full copies.

`keep` turns identical and delta records into full copies: it rebuilds each file under `originals/`, checks its SHA-256 against the record, updates the record and removes the delta it replaced. Afterwards the reference files can go without loss. Like `drop`, it lists first and acts with `--yes`, and each keep is recorded in `ARCHIVE.json` under `keeps`.

Plain SQL reaches everything else. Open the databases with `immutable=1`, which writes nothing:

```
sqlite3 "file:ARCHIVE/bytes/recovery.sqlite3?immutable=1" \
  "SELECT path FROM files WHERE json_extract(record, '$.source_identity[1]') > 100000000"
# sources over 100 MB
sqlite3 "file:ARCHIVE/metadata/differences.sqlite3?immutable=1" "SELECT * FROM
change_counts LIMIT 20"
```

`bytes/recovery.sqlite3` holds `files(path, record)`, `versions(path, version, record)`, `errors`, and after a drop `trash` and `dropped`; a record is the JSON object shown under *What a record holds*. The metadata index's tables are described in Part V, *Full metadata snapshots and queries*.

Part II: The archive

Layout

An archive is one directory with up to three sections and its own record:

- **`report/`**, the comparison: `pairs.jsonl` with one result per pair, `summary.json`, and `cache.sqlite3`, the cache a repeated run reuses.
- **`bytes/`**, the preservation: `recovery.sqlite3` with one record per file, earlier versions and errors; `deltas/` and `originals/`, the payloads; `manifest.jsonl`, `inventory.json`, `directories.json` and `summary.json`.
- **`metadata/`**, the searchable metadata: `batches/*.jsonl.gz` with each file's full record on both sides, `differences.sqlite3`, the index `query` reads, `summary.json` and `provenance.json`.
- **`ARCHIVE.json`**, every run's options, sources, implementation and exit statuses, every migration, drop and keep; and **`MIGRATION.jsonl`**, the steps of a migration if there was one.
- **`skeletons/`**, after `dngc skeleton build`: `skeletons.sqlite3`, a few kilobytes per reference file, from which a reference that has since been edited is rebuilt as it was (Part VII).
- **`trash/`**, after a `drop --yes`: the dropped records' payloads, one folder per drop, until it is restored or emptied.

Records name their deltas and full copies relative to the byte section, so the archive can move as a whole.

The byte-section rule

Every tool that writes or reads preserved bytes finds the byte section by one rule, implemented alike in Python, OCaml and Rust:

1. if `ARCHIVE/recovery.sqlite3` exists and `ARCHIVE/bytes` does not, the byte section is `ARCHIVE` itself (an archive written before this layout);
2. else, if the path given ends in `bytes`, it is that path, never `bytes/bytes`;
3. else it is `ARCHIVE/bytes`, created for a new archive.

Metadata, provenance, Semantic Unison's pair lists, `report/` and `ARCHIVE.json` stay at the archive's top. Writers print the byte section they chose (`Byte section: PATH`) before preserving.

Semantic Unison writes the same layout when it preserves what a shuttle disk brings, so one archive can hold both a `dngc` run and Unison's shuttle cycles.

What a record holds

A record in `bytes/recovery.sqlite3` is a JSON object. For an identical file it looks like this:


```
{
  "relative_path": "2021/October/A1000946.DNG",
  "source": "/Volumes/Archive/2021/October/A1000946.DNG",
  "reference": "/Volumes/Backup/2021/October/A1000946.DNG",
  "source_identity": [1864268, 26214400, 1637379572000000000, 1698199405000000000],
  "reference_identity": [883821, 26214400, 1637379572000000000, 1767774679100649765],
  "source_attrs": {"mode": 420, "uid": 501, "gid": 20, "mtime_ns": 1637379572000000000,
                   "atime_ns": 1793755763000000000, "flags": 0, "xattrs": {}},
  "source_sha256": "…", "reference_sha256": "…",
  "original": null, "whole_file_equal": true,
  "preserved_bytes": 0, "reconstruction_verified": true,
```

```
  "reference_relative_path": "2021/October/A1000946.DNG"
}
```

The identities are inode, size, modification time and change time. A delta record adds `delta`, `delta_sha256` and `delta_format`; a full copy names its file under `original`. `restore` needs only this record, the payload it names, and for a delta or an identical record the reference file, or an edited version of it and its skeleton.

An archive is faithful to what it was given. An empty file is recorded as empty, with the SHA-256 of zero bytes; a damaged file is preserved damaged. Finding damage is a comparison's job, and Part III shows why.

The metadata section

`metadata/batches/` keeps each file's full metadata record on both sides of a pair, and `differences.sqlite3` indexes one row per changed tag. It is written by a comparison with `--metadata exiftool`, by `--metadata phindex` from phindex's records, or by Semantic Unison's semantic diff.

Engines are never mixed. Perl ExifTool is the engine of record. Records from the exiftool-rs engine carry `ExifTool:DngCompareEngine` and are a separate identity; a tool asked to add records of one engine to an archive holding the other refuses. phindex's records are exiftool-rs records, so an archive whose metadata came from Perl ExifTool cannot take `--metadata phindex`.

Records made from phindex keep strings over 1 KiB, such as embedded thumbnails and profile tables, as `long-sha256:<hex>:<length>`. They still show that such a value differs, not how. The bytes themselves are in the byte section.

Kinds of archive

Before `dngc`, each tool wrote its own kind of output. All of them are still read:

Kind	Holds	Rebuilds files?	Searchable?
A. Comparison report	per pair: raw and preview equal or different, metadata differences	no	by reading JSON
B. Byte preservation	every source file as a reference, a verified delta or a full copy	**yes**, byte for byte	no
C. Metadata section	each file's full record on both sides, plus a typed difference per changed tag	no	**yes, SQL**
D. Year archive	B and C per year, plus exact XMP packets and image classification	yes	yes
E. XMP packet archive	exact embedded XMP packets with SHA-256	the XMP only	no
F. Develop-settings archive	develop settings captured from several editors	no	yes
G. phindex index	every file's identities and a shortened metadata record	no	yes, per file

A `dngc` archive is A, B and C in one directory. Only B and D can rebuild a file.

Migrating an older archive

```
dngc migrate /Volumes/Archive/old-archive --dry-run          # show the plan
dngc migrate /Volumes/Archive/old-archive --report DIR       # move bytes/, copy a report
in
dngc migrate /Volumes/Archive/old-archive --undo             # put it back
```

`migrate` moves exactly the entries the byte tools wrote at an older archive's top into `bytes/`, by renames on the same volume. It never copies and deletes. It refuses while any tool holds the archive's locks, or when a database has uncheckpointed changes. Every step is logged to `MIGRATION.jsonl` before it happens, so an interrupted migration resumes and `--undo` reverses it. Afterwards it runs `verify --restore-sample 3`. A metadata-only archive gets its `ARCHIVE.json` and nothing moves. Part V's *Recovery after a failure* covers older archives in more depth.

Shuttles: wiped and refilled sources

A shuttle disk is emptied after its files are archived, then filled again, sometimes with other files under the same relative paths. The archive keeps its history through that:

- **Reading needs no source.** `status`, `verify`, `query`, `versions` and `restore` read only the archive and, for a delta, the reference.

- **Earlier records are never deleted by a run.** A run checks only its own files at the end; records of earlier runs stay as they are. Only `dngc drop` removes records, and only when told to.
- **A missing source is reported, not an error in the archive.** A listed source that is gone is printed as `MISSING REL` and counted in `missing_sources`, and its earlier record stays.
- **A refilled path is versioned.** When a path's content changes, its earlier record moves to the `versions` table under its label, and its payload stays where it was. The new content's payload goes under `versions/LABEL/`, so no payload is ever overwritten.

Reading leaves the archive as it was

A plain read-only open of a SQLite file in WAL mode creates `-wal` and `-shm` files beside it, and on macOS fails without them. Every reading verb therefore opens the archive's databases with SQLite's `immutable=1` when no writer holds the archive, and otherwise reads a private copy. An archive on a backup volume, or one Semantic Unison is mirroring, is not changed by being looked at.

Part III: Why preservation is the default

The first edition's main command compared, and preservation was a separate step. A comparison says what differs; only an archive keeps it. Now `dngc run` preserves unless told `--bytes none`, for three reasons met in practice.

Raw-equal is not a backup. On one real archive, 480,069 pairs of DNGs had identical raw samples on both volumes, yet every one of them differed somewhere else: XMP, previews, develop history. A deletion based on raw equality would have lost all of that.

A preview can look fine while the file is broken. On the same archive, 77 original DNGs had been cut short by an interrupted copy years earlier, while an older converter's output of the same photographs was intact on another volume. Thumbnailing tools render a DNG's embedded preview, which sits early in the file and survived the truncation, so the damaged files looked normal in a browser. Only a comparison of the raw pixels showed them. A comparison-only run that had deleted the "duplicates" would have kept the damaged copies and lost the good ones. With an archive, both are recoverable whichever is kept.

Files nobody listed are often the unique ones. A pair list from an alignment holds the pairs that differ. The files with no counterpart (sidecars, a renamed export, a temporary file) are exactly the ones whose only copy is on the source. A whole-tree `dngc run` covers them as full copies.

The cost is small. Deltas against a matching reference are compact: on the archive above, half a million pairs needed about 10 GB of deltas for about 18 TB of source files, roughly 0.06 per cent.

Part IV: Implementations and performance

Three implementations, one archive

Implementation	Command	Notes
OCaml and OxCaml	`dngc`, `dng-archive all`, `dng-archive preserve`	the default; one native binary
Rust	`dng-compare-all`	`dngc run --impl rust`
Python	`compare_dng.py`, `preserve_apo.py`, `phindex_metadata.py`	the reference; `dngc run --impl python`

All three read and write the same files: the same report, cache keys and stamps, the same records field for field, the same deltas. Either can resume what another started and restore what another preserved. `test_native_all.py` runs the Python reference and each native implementation on the same synthetic pairs and requires equal outputs; `test_dngc.py` does the same for both front ends of `dngc`.

One read per pair

The native `all` command reads each DNG pair once: both files whole into memory, then the comparison, the hashes, the metadata and the delta come from those bytes. The only further disk reads are ExifTool's, and with `--metadata phindex` there are none. Part V, *Rust and OCaml extraction pools* and the *One-pass* notes in the repository give the per-pair procedure.

Measured on one archive

These figures come from one whole-tree run on 2026-11-08 and 9: 1.71 million source files on a four-disk RAID, a reference of 1.63 million files on a RAID enclosure that reads about 240 MB/s, an Intel Mac, OxCaml 5.4 build.

Step	Measured	Before the fix that made it so
Walking the source tree (1,709,944 files)	44 s	
Archive setup for 713,261 DNG pairs	12 s	about 10 minutes: a `realpath` per file instead of per directory, at 120 a second on the slow enclosure
Identical DNGs recorded in bulk	226,005 in 127 s	a few a second, each through a worker process
Cached DNG pairs	about 350 pairs a second; 1,560 cold with the disks otherwise idle	none reused: a remount had changed the device number in every cached stamp
Metadata section, 1.7 million pairs	about 25 minutes	1 h 50 min: every pair's two records parsed and stored, including the byte-identical ones
Resuming the metadata step	about 20 s, 3.2 GB	7 minutes and tens of gigabytes: every batch parsed and kept in memory
`verify` of a copy on hard disks	minutes	most of an hour: the index's integrity check read the disks page by page

The lesson in each row is the same: on hard disks behind a slow enclosure, a cost per file, however small, multiplied by a million files decides the run. `dngc` now resolves each directory once, writes identical records without worker processes, and does not let a remount invalidate its cache.

Part V: The underlying tools

The chapters below are the first edition's, kept for the tools `dngc` runs and for the workflows it does not cover yet: full copies of a whole tree (`preserve_apo.py --full-copy`), year archives (`preserve_years.py`) and exiftool-rs metadata extraction (`portable_metadata.py`). Their commands still work as described.

Install and build

The Python comparison needs Python 3.12 or newer, ExifTool, NumPy, tifffile and imagecodecs. The project pins its dependency set through uv. Native Rust targets Unix and is pinned to Rust **1.99.0, edition 2024**. The metadata pool is written in ordinary OCaml. Its historical executable name ends in `-oxcaml` so existing archives and backend selection remain compatible; that name does not require an OxCaml compiler. The OCaml byte worker's C SHA binding uses macOS CommonCrypto.

On macOS, install uv and rustup using their official installation instructions, then install the remaining tools and build:

```
brew install exiftool xdelta ocaml
git clone https://github.com/firstpair/dng-compare.git
cd dng-compare
make setup
rustup toolchain install 1.99.0
```

```
make build
make test
```

Run these from the real repository root. `make setup` uses `uv sync`; ordinary Python commands use `uv run python`. Do not substitute a different Rust release without recording and testing that change. Confirm the compilers you actually use:

```
uv run python --version
rustc --version
ocamlopt -version
exiftool -ver
xdelta3 -V
```

`make build-rust` builds the ordinary Rust binaries. `make build-oxcaml` builds `metadata-extract-oxcaml` and `byte-verify-oxcaml`. The Makefile defaults to `OCAMLOPT=ocamlopt` and accepts an explicit compiler or flags override:

```
make build-oxcaml OCAMLOPT=/path/to/ocamlopt OCAMLFLAGS=-O3
```

The older `OXCAMLOPT` override remains available for an existing switch. Linux can use the Python comparator and Unix Rust tools when their dependencies are available; the macOS CommonCrypto OCaml byte binding needs a platform-specific replacement there. Native Windows support is not supplied by these Unix file, process and lock APIs.

The tests create temporary synthetic files. They check comparison, cache invalidation, preservation, bounded parallel work and checkpoint behavior. A passing fixture suite is evidence for those cases, not validation of every camera, codec or DNG layout. Optional Turso probes are separate from the normal build and are discussed below. The publication validation record names the observed compilers, fixture results and remaining untested scopes.

A reproducible synthetic example

This example creates two small DNG-like TIFF files in a dedicated directory. They contain the same 16-bit grayscale samples but different ImageDescription metadata. They are test fixtures rather than certified camera originals.

Start in the repository root, with a new empty `demo` directory. The script refuses to reuse an existing directory:

```
uv run python - <<'PY'
from pathlib import Path
import numpy as np
import tifffile

root = Path('demo')
root.mkdir(exist_ok=False)
(root / 'left').mkdir()
(root / 'right').mkdir()
pixels = np.arange(256, dtype=np.uint16).reshape(16, 16)
for side, description in [('left', 'Older metadata'),
                          ('right', 'Newer metadata with a longer description')]:
    tifffile.imwrite(root / side / 'same.dng', pixels,
                     description=description, metadata=None,
                     extratags=[(50706, 'B', 4, (1, 4, 0, 0), False)])
```

```
PY
uv run python compare_dng.py demo/left demo/right --output demo/report
```

Expected results are complete raw coverage, equal auxiliary coverage for this simple fixture, and an embedded ImageDescription difference. The image method should be `encoded_blocks`: different metadata placement has not required sample decoding. Read the actual report rather than relying on this description:

```
uv run python - <<'PY'
import json
from pathlib import Path
s = json.loads(Path('demo/report/summary.json').read_text())
p = json.loads(Path('demo/report/pairs.jsonl').read_text().splitlines()[0])
print(s['complete'], s['left_subsumed_by_right'])
print(p['raw'])
print(p['differences']['embedded'])
PY
```

Run the same comparison again. It should reuse the successful pair if both file stamps are unchanged. Run the Rust scanner against the same generated roots:

```
./rust-scanner/target/release/dng-subset \
  demo/left demo/right demo/rust-report --rate-mib 32
```

To exercise decoded comparison, create another new fixture using the same array but `compression='deflate'` and a different `rowsperstrip` in one copy. The Python comparator should decode and compare the complete arrays. Rust may leave that pair unverified because it does not have a decode fallback. The test suite includes that case and a deliberately changed sample and truncated file.

This is also a suitable input for a preservation rehearsal. Store a delta and restore the older bytes into a new destination:

```
uv run python preserve_apo.py scan demo/left demo/right demo/archive \
  --delta --workers 1 --rate-mib 32
uv run python preserve_apo.py restore demo/archive same.dng demo/restored.dng
./rust-scanner/target/release/byte-verify compare \
  demo/left/same.dng demo/restored.dng
```

The last line should print equal source and restored SHA-256 values followed by `1`. Keep the reference fixture: the delta depends on its exact bytes. Use `--full-copy` in a separate archive if you want a self-contained original instead.

Python comparison command

```
uv run python compare_dng.py LEFT RIGHT [options]
```

Option	Default	Purpose
`--output DIRECTORY`	`dng-report`	Reports and comparison checkpoint
`--workers N`	2	Concurrent image pairs; must be positive
`--batch-size N`	32	Candidate pairs per metadata/checkpoint batch
`--sample N`	Full candidate set	Evenly spaced candidate pairs
`--limit N`	Full candidate set	First N candidate pairs
`--fresh`	Reuse eligible cache	Ignore saved comparison results
`--pair LEFT RIGHT`	Name matching	Compare LEFT with RIGHT; repeatable
`--pairs FILE`	Name matching	Read `LEFT<TAB>RIGHT` pairs, one per line

`--sample` and `--limit` are mutually exclusive. Their unit is a **candidate pair**, not necessarily a unique source image. If one basename has several copies on each side, sampling may select multiple candidates for the same left file and leave other left files untouched.

The program walks both roots recursively, includes extension-case variants of `.dng`, and ignores macOS `._` companion files. Basename matching itself is case-sensitive. Each left physical path is a coverage obligation. For a basename appearing several times, all candidates are scheduled, with the same-relative-path candidate first. Additional right files are ignored. Unreadable directories fail rather than silently disappearing from the scope.

This is a one-way containment check, not a two-way tree diff. To check the reverse direction, use a separate report directory and reverse the roots. A renamed DNG is missing under this matcher even if its bytes exist elsewhere. Use an explicit mapping or an independent identity investigation before treating that as actual original loss.

An explicit mapping replaces name matching for each listed left file; unlisted left files are still matched by name. Give pairs with `--pair LEFT RIGHT` (repeatable) or in a `--pairs` file of `LEFT<TAB>RIGHT` lines, where blank lines and lines starting with `#` are ignored. Each member is an absolute path, a path relative to its own root, or a basename that occurs exactly once under that root. It must be a DNG the inventory found under its root; anything else, or an ambiguous basename, is an argument error (exit 2). Several right files may be listed for one left file, and each is a candidate. `summary.json` records the number of listed pairs as `explicit_pairs`, and `left_only` covers the left files that have no pair of either kind. Root membership and filename matches are candidate selection, never the final image proof.

How the Python image comparison works

The comparator requires a DNGVersion tag in each first page and traverses main and subordinate TIFF pages. It classifies primary raw pages using subfile type zero and its supported photometric codes; other pages go into the auxiliary group. It compares corresponding pages in their traversal order. Different page counts remain unverified instead of being guessed into correspondence.

For each page it validates the declared strip/tile ranges against the physical file length and reads every payload block. SHA-256 includes each block length before its bytes, so block boundaries are part of the encoded comparison. A matching payload digest is accepted on the fast path only with the selected encoding signature, including byte order and the implemented interpretation parameters.

If that path does not match, `tifffile` and `imagecodecs` decode the two pages. Shapes and complete arrays must compare equal; NaNs compare equal for this purpose. A changed decoded array is `different`. A missing decoder, malformed page or decode error is `unverified`, with its actual error retained. Differing compressed bytes alone are not classified as a pixel difference.

Here, raw samples mean the stored sensor or linear samples, including the sample margins, before rendering. Crop, white balance, orientation, DNG opcodes, profiles and develop instructions are not applied. Equal samples are therefore compatible with different appearance in Lightroom. The comparison is not an Adobe SDK validation, a demosaicing test, or proof that an identical damaged pair is healthy. `OriginalRawFileData`, which can contain an embedded original source file, is excluded from image payload and ExifTool comparisons; use whole-file preservation when that content must be retained.

Metadata extraction and categories

The comparison invokes ExifTool once per pending batch, overlapping extraction with image work. It requests JSON, grouped/instance-qualified names, numeric output, structured values, unknown tags and binary contents. Preview/raw image payloads are excluded because the image comparator reads those directly. The portable snapshot route additionally retains the wider grouped output described under metadata archives.

ExifTool's JSON and grouping options are important: structured XMP and binary values can be retained instead of a flattened presentation or a byte-count placeholder. The tool's numeric output is its own representation of metadata; Python does not recover the original TIFF storage type from that JSON. ExifTool's application documentation.

The main comparator normalizes global `CopyN` group numbering so adding one XMP field does not make unchanged EXIF duplicate instances appear changed. Real duplicate values within a group remain retained. It flattens dictionaries for field comparison and labels differences directionally:

- `added`: present on the right and absent on the left.
- `removed`: present on the left and absent on the right.
- `changed`: present on both sides with unequal values.

It separates `embedded`, `filesystem` and `layout`. Directory, filename, access time, ExifTool's own fields and derived Composite fields are excluded from this comparison. File size, modification/change time and permissions can appear as filesystem differences. TIFF image offsets/counts and preview placement belong to layout. A layout difference need not change samples.

Metadata comparison covers what ExifTool exposes under the chosen flags. It does not compare all arbitrary padding, unused bytes or uninterpreted private structures. Warnings and errors are retained. In the compact Python report, large differing values have a canonical JSON length, SHA-256 and prefix instead of the full printed value. The portable gzip archive retains full extracted values for later queries or restoration decisions.

Interpret the Python reports

`summary.json` is updated after each completed batch. Its final `complete` flag means all candidate pairs in the full inventory were processed. It can be true while some left files are missing or different. Missing filenames have no candidate pairs, so an empty candidate workload is not proof of coverage of a nonempty left root.

`left_files_covered` counts unique left paths, separately for raw, all-images and metadata preservation. `left_subsumed_by_right` compares each count with the entire left inventory:

Field	Acceptance
`raw`	At least one right candidate has equal raw pages
`all_images`	A matching candidate has equal raw and auxiliary pages
`metadata_preserved`	A raw-equal candidate has successful metadata extraction, no warnings, and no removed or changed embedded fields

Added embedded metadata is allowed by `metadata_preserved`. That field does not include arbitrary uninterpreted bytes or guarantee that the auxiliary images are equal: these are independent conditions. The qualifying candidate may differ between coverage categories. Consult the individual pair records to identify a specific copy satisfying all the conditions you need.

`counts` reports pairs and verdicts, not unique image coverage. A duplicate-name workload can have both `raw_different` and `raw_equal` pairs while the left file is fully covered by one equal candidate. `metadata_patterns` groups recurring sets of changes; `metadata_tags` includes frequencies and at most three examples per changed field. `verified_payload_bytes` is aggregated from pair results, including reused results; it is not necessarily physical I/O in this invocation.

`pairs.jsonl` is the detailed evidence. `unmatched.json` lists missing left names and all duplicate-name candidate sets. At normal completion, `not-confirmed.json` lists left paths without a confirmed raw copy, including untested paths from a sample. It can be absent or stale after an interruption, so use the current summary and pair/cache evidence in that case.

Exit status is 0 for complete left raw coverage, 1 for unconfirmed coverage, 2 for argument errors, and 130 for a caught keyboard interruption. Metadata changes alone do not fail the raw verdict. Read failures and source changes remain errors, rather than equal results.

Resume and protect a comparison

The Python checkpoint key binds cache version and both absolute paths. Reuse requires matching device, inode, size, nanosecond mtime and ctime on both sources. Successful metadata/image results can be cached; errors and unverified raw or auxiliary groups are retried. Rerunning rebuilds the JSON reports from the current inventory and reusable checkpoint.

A filesystem identity check avoids rereading unchanged files, but it is not a cryptographic freshness guarantee against a privileged actor who can replace bytes while preserving all recorded facts. If the audit requires fresh payload reads, use `--fresh` or a new report directory. A remount can invalidate the Python device stamp and cause a reread.

The Python comparator holds a nonblocking Unix lock for its output directory. Do not edit source metadata, replace images, alter the report database or start a second owner while it runs. Read-only access can still

update filesystem access times; that is excluded from verdicts. Keep output directories outside the photo roots to avoid accidentally treating reports as source material in later archive commands.

Rust bounded scanner

```
dng-subset LEFT RIGHT REPORT [--rate-mib N] [--max-new N]
           [--import-cache FILE] [--after SUMMARY]
```

The binary is `rust-scanner/target/release/dng-subset`. It has one process and checks one pair at a time using 1 MiB buffers. It recognizes classic TIFF and BigTIFF headers, traverses implemented IFD/subIFD paths and confirms equal selected raw blocks plus their encoding tags. It uses direct byte comparisons, not stored DNG digests. The parser has finite limits for IFD counts, tag counts, selected tag sizes and integer arrays. Unsupported or damaged structures remain unverified unless its exact whole-file fallback proves that the bytes are preserved.

The default `--rate-mib 32` accounts for combined raw-data reads in MiB/s; choose any finite number above zero through 512. It is an average pacing budget, not a kernel I/O ceiling for all metadata, filesystem and decoder traffic. `--max-new N` limits newly checked left files, leaving others pending. Matching names and relative-path priority follow the same basic one-way model as Python.

Unlike Python, Rust does not decode different compression/layouts or compare auxiliary previews and metadata. `not-confirmed.tsv` reports `MISSING`, `UNVERIFIED` and `PENDING`. A differing encoded block is unverified, not a proven sample difference. Exit 0 confirms complete raw subsumption; exit 1 leaves coverage unconfirmed; parsing/argument/I/O failures are nonzero as well. Always inspect `summary.json`, which retains `total_left`, `verified_equal`, `missing_name`, `unverified`, `complete` and `left_subsumed_by_right`.

`verified.tsv` is append-only. Its four columns are hex-encoded left path, hex-encoded right path and comma-separated stamps for both files. Each stamp has device, inode, size, mtime and ctime. Resume retains path membership and checks inode, size, mtime and ctime, deliberately ignoring device number across external volume remounts. It does not hash accepted payloads again. There is no cache version/executable hash bound to this TSV format and no automatic report lock; keep the producing source version and one exclusive owner yourself. A new report directory gives a fresh scan.

`--import-cache` trusts already verified records in the same format after current identity checks. It is not a way to admit arbitrary filename matches. The `export_verified.py` helper exports eligible Python raw-equal cache results for explicit roots. Keep its input report and tool provenance.

`--after SUMMARY` waits until the named file contains the literal `"complete": true` and both input directories exist, checking once per minute. It does not parse and require a successful subsumption verdict. A completed prior scan can have missing or unverified files. Use this only as a scheduling aid, not a safety dependency, and do not share report directories across jobs.

Rust metadata reporter

```
dng-metadata IMPORTED_VERIFIED.tsv RUST_VERIFIED.tsv REPORT_DIR BATCH_SIZE
```

This ExifTool reporter reads verified pair ledgers, deduplicates by left path and writes atomic per-batch JSONL files plus `summary.json` and a combined `pairs.jsonl`. `BATCH_SIZE` is 1–64. If there is no imported ledger, supply an existing empty file:

```
mkdir -p reports
touch reports/empty-import.tsv
```

```
./rust-scanner/target/release/dng-metadata \
  reports/empty-import.tsv reports/rust/verified.tsv reports/rust-metadata 32
```

It compares values in full but shows large values with a length and prefix; its compact representation does not include the Python compact-value SHA. It has its own report schema, with `changes` and `status`, rather than Python's `differences` and `metadata_status` fields.

The reporter does not reverify payload equality or current source identity. Existing batches are reused using their index and record count, with no complete manifest/content guard. Keep the pair ledgers, batch size and sources unchanged; use a new output directory after any change. A completed metadata summary can include errors. This reporter is useful for stable, already qualified pairs; the full portable archive is the stronger provenance route.

Archive embedded XMP bytes

```
./rust-scanner/target/release/dng-subset --archive-xmp \
  /media/source/foto archives/source-xmp
```

This extracts TIFF XMP tag 700 bytes under `packets/`, preserving the relative source layout. It records packet size, SHA-256, extraction method and source identity in manifests. On a parser failure it can use ExifTool extraction as a fallback, recorded separately. It does not modify a source DNG.

Per-batch manifests permit resume after checking source stamps and stored packet hashes. The final summary counts files with/without XMP, packets and errors and reports `all_xmp_saved`. A file with no packet is different from a failed extraction. Existing sidecars are outside this command; preserve them separately. Embedded source XMP is not automatically a current cloud develop packet, and a packet hash must never be renamed to impersonate a different requested hash.

Preserve exact original files

The historical filename `preserve_apo.py` now accepts explicit generic roots:

```
uv run python preserve_apo.py scan SOURCE REFERENCE ARCHIVE
    [--project DIRECTORY] [--workers N] [--rate-mib N]
    [--full-copy] [--delta] [--pairs FILE]
    [--source-index PHINDEX.sqlite --reference-index PHINDEX.sqlite]
uv run python preserve_apo.py restore ARCHIVE RELATIVE_PATH OUTPUT
    [--reference-root DIRECTORY]
```

The scan inventories **all regular files**, not just DNGs, and rejects unsupported file entries such as symlinks. It uses any supplied known-pair ledger and otherwise looks for the same relative path under the reference. An absent reference is preserved as a complete original. Keep source/reference roots stable, writable output separate, and one archive owner at a time.

Mode	Equal whole-file bytes	Different bytes with reference	Missing reference
Default	SHA-bound reference	Complete original	Complete original
`--delta`	SHA-bound reference	Verified VCDIFF delta	Complete original
`--full-copy`	Complete original	Complete original	Complete original

`--full-copy` takes precedence over delta storage. The scan compares complete source/reference bytes, records their SHA-256, validates before/after identities, and rereads saved complete originals. For a delta,

xdelta3 encodes the source version against the reference. The archive then decodes it and requires the reconstructed SHA to equal the observed source SHA. VCDIFF is a standard delta format, rather than an image-specific edit representation. RFC 3284.

With phindex indexes of both trees, a pair whose current index rows have equal SHA-256 is recorded as whole-file equal without reading either file; a row is current when its size and modification time match the file now.

`--pairs FILE` preserves only the listed files instead of the whole source. Each JSON line is `{"source": PATH, "reference": PATH}`; the reference may sit under another relative path, and both must lie under their roots. Each run adds to the same archive. Semantic Unison's `unison -semdiff -preservefull` writes these lists for files whose image is identical but whose metadata differs, and preserves them with `dng-archive preserve` (`ocaml-archive/`), the OCaml port of this `--pairs` mode, which writes the same records and deltas: either tool restores what the other preserved.

The scan records source attributes, including mode, times and extended attributes where supported. A restore verifies payload bytes before a no-overwrite commit and reapplies saved xattrs, mode and times. It does not recreate every possible filesystem property: saved uid/gid/flags are not all automatically restored. Cross-platform xattr support also differs. Preserve that distinction from an exact file-payload guarantee.

The public entry points reject an archive output that overlaps a source or reference root. This protects against writing recovery files into the trees being audited. It does not supply an automatic cross-process lock to every low-level API. Use one archive owner and separate output directories.

`summary.json` distinguishes processed completion, errors and `byte_preservation_complete`. `recovery.sqlite3` is the checkpoint, `manifest.jsonl` is the portable record view, `inventory.json` covers source files, and `directories.json` retains directory attributes. Each recovery record says whether it uses `original`, `delta` or a reference alone. The scan never sets `safe_to_delete_source` true by itself.

Default and delta archives depend on retained exact reference bytes. An XMP write can break that dependency despite unchanged samples. Keep another independent reference copy, archive the reference too, or choose full-copy storage. Restore can accept a relocated reference root through `--reference-root`; the relative reference path must be safe and the bytes must still match the recorded hash. An existing output is rejected, so restore into a new location.

Native byte workers

The Python archive layer can select a native worker with `DNG_BYTE_BACKEND`:

```
DNG_BYTE_BACKEND="$PWD/rust-scanner/target/release/byte-verify" \
uv run python preserve_apo.py scan /media/source/foto /media/reference/foto \
  archives/source --delta --workers 2 --rate-mib 64
```

Use `$PWD/byte-verify-oxcaml` to select the OCaml worker on macOS. Both support:

```
hash FILE
compare SOURCE REFERENCE
delta REFERENCE SOURCE DELTA EXPECTED_SOURCE_SHA256
```

`compare` prints source SHA-256, reference SHA-256 and equality `1` or `0`, tab separated. An empty reference argument requests source hashing without a reference. `delta` outputs its own SHA only after reconstruction passes. Workers own bounded buffers and the external xdelta process; Python owns file identity, inventory, checkpoint and acceptance. Direct worker calls do not supply those wider guards. Native byte operations are whole-file operations, not stored-sample or Lightroom-native fingerprint calculations.

Full metadata snapshots and queries

```
uv run python portable_metadata.py SOURCE REFERENCE ARCHIVE
    [--project DIRECTORY] [--batch-size N]
```

The portable snapshot route retains unabridged ExifTool JSON in atomic `metadata/batches/*.jsonl.gz`, independently reads it back, and records batch checksums. It inventories files and before/after identities. `.DS_Store` has an explicit ignored-system classification instead of a metadata failure. Missing references remain visible and do not produce a fictitious comparison.

The original archive schema uses names `apo` for **source** and `bumer` for **reference**. Those field names are retained for compatibility; they do not select a machine or drive. The directional change labels mean:

Historical label	Generic meaning
`added_in_bumer`	Added in the reference
`missing_in_bumer`	Missing from the reference
`changed_in_bumer`	Source and reference extracted values differ

The snapshot keeps wider `-a -G0:1:4 -n -struct -b -u -U` ExifTool output and rejects duplicate JSON keys. It retains arrays, structures and binary metadata values, excluding image payloads and embedded OriginalRawFileData. Its filesystem classification and field names differ from the compact comparator; do not join the two schemas by assuming identical keys.

`metadata_index.py ARCHIVE` creates/rebuilds the searchable `metadata/differences.sqlite3` index from gzip batches. It deduplicates full JSON values by SHA and stores compressed complete values, with coverage and change views. Query only a completed checkpoint after its writer closes:

```
uv run python metadata_index.py archives/source
uv run python - <<'PY'
import sqlite3
from pathlib import Path
p = Path('archives/source/metadata/differences.sqlite3').resolve()
with sqlite3.connect(f'file:{p}?mode=ro', uri=True) as db:
    for row in db.execute('SELECT tag, kind, affected_files FROM field_changes '
                          'ORDER BY affected_files DESC LIMIT 20'):
        print(row)
PY
```

Metadata snapshots are descriptive, not a merge or write-back plan. An added field can be harmless, a timestamp can be bookkeeping, and a rating or crop can be a meaningful user choice. Inspect exact values and decide what to retain.

Develop settings across editors

The metadata snapshots above compare two copies of the same files. A different question is how several editors set up the same photographs, for example after running Auto in Lightroom, Lightroom Classic and Capture One. `develop_settings.py` stores each editor's settings as a separate, write-once **capture** in one archive:

Source (`app`)	Reads	Raw record kept
`xmp`	Settings embedded in each DNG's XMP, or in exported JPEGs with `--extensions jpg` (or `dng,jpg`), via ExifTool; or an existing `compare_dng.metadata` JSONL snapshot (`--snapshot`)	All ExifTool tags
`lrcat`	A Lightroom Classic catalog, opened read-only with the standard SQLite driver	Settings text, digest, virtual-copy name, history step names
`capture-one`	Capture One `.cos` settings files (default `FOLDER/CaptureOne/Settings*`)	Complete XML

```
uv run python develop_settings.py capture ARCHIVE lr xmp FOLDER
uv run python develop_settings.py capture ARCHIVE lrc lrcat FOLDER \
  --catalog CATALOG.lrcat
uv run python develop_settings.py capture ARCHIVE c1 capture-one FOLDER
uv run python develop_settings.py compare ARCHIVE lr lrc
uv run python develop_settings.py compare ARCHIVE lr c1 \
  --map XMP-crs:Exposure2012=CaptureOne:AL:Exposure
```

Each capture writes `ARCHIVE/settings/LABEL.jsonl.gz` (one record per photo: photo name, source identity, typed `settings` and the unchanged raw source) and `LABEL.provenance.json` (schema, source, photo count, record SHA-256, tool commit). A label is never overwritten. Typed settings use one namespace: Adobe settings as `XMP-crs:<name>`, whether they came from XMP or a catalog, and Capture One settings as `CaptureOne:<block>:<name>` (for example `DL` for the main adjustments and `AL` for the block holding automatic values). Text that is exactly a number or boolean is stored as one; everything else stays text.

`settings.sqlite3` indexes every typed value; the `setting_values` view lists `label, app, photo, tag, value_json, number`. `reindex` rebuilds it from the captures after checking each record SHA-256. `compare` reports, per tag, the photos present in both captures, how many are identical (numbers by value), and for numbers the median and 90th-percentile differences and the correlation.

There are two implementations with the same command line and archive format:

	Python `develop_settings.py`	Rust `develop-settings`
Reads XMP with	ExifTool (`compare_dng.metadata`)	The XMP packet in a DNG's first IFD or a JPEG's APP1 segments, parsed directly
Records and provenance	Pydantic models	serde types
`raw` for `xmp`	ExifTool tags and warnings	The XMP packet text

Build the Rust one with `make build-rust` (or `cargo build --release --bin develop-settings` in `rust-scanner`) and run it as `rust-scanner/target/release/develop-settings capture ...`. Both type values the same way, name settings by their XMP property (ExifTool's `ColorTemperature` is recorded as `XMP-crs:Temperature`, its XMP name and Lightroom Classic's), flatten structures to `Parent/Field` and write the same canonical `value_json`, so either can add captures to an archive, reindex it or compare it.

`test_develop_settings.py` checks that both write equal records from the same fixtures and that each reindexes and compares the other's captures identically.

Settings in rendered JPEGs

Exported JPEGs are a second place to find settings, and often the only record of what an export actually used:

Rendering	Settings in the JPEG
Lightroom and Lightroom Classic exports	The complete Camera Raw settings (`crs:`), including the Auto marker when Auto was applied
Capture One exports	None; read the RAW folder's `.cos` files instead
Out-of-camera JPEGs	Only EXIF basics such as white balance mode, contrast, saturation and sharpness

```
uv run python develop_settings.py capture ARCHIVE lr-export xmp EXPORT_FOLDER \
  --extensions jpg
```

A capture records which extensions it read in its provenance. The Rust reader takes the XMP from the JPEG's APP1 segments, including Adobe's extended XMP, whose chunks it assembles by GUID and offset. Exports are a good cross-check: in one 446-photo test, the settings embedded in Lightroom's Auto exports matched the settings Lightroom had written into the DNGs on every photo, and the Lightroom Classic exports matched its catalog.

Capture One's `.cos` files hold the current state, including any manual adjustment made after Auto; compare their modification times with the export times before treating them as the settings of an older export.

Limits:

- An `xmp` capture reads the files and fails if one changes during the read. The Rust reader reads the XMP packet stored in a DNG's first IFD or a JPEG's APP1 segments.
- Values are typed exactly: the Rust build parses floats with serde_json's `float_roundtrip`, so every stored number reads back bit for bit.
- Lightroom Classic computes Auto lazily and stores `-999999` placeholders until a photo is rendered. `lrcat` refuses such photos unless `--allow-pending` is given, and then records the placeholders with a note.
- Editors use different controls and scales. `--map` pairs tags; it does not claim that the controls are equivalent.
- Captures describe settings; they do not reproduce rendered appearance and nothing is written back to photos, catalogs or settings files.

Rust and OCaml extraction pools

`metadata-extract WORKERS` reads a JSON array of UTF-8 paths from stdin. The OCaml equivalent `metadata-extract-oxcaml WORKERS` reads NUL-delimited paths. Both emit ExifTool JSON records in input order without deserializing and reserializing their metadata values in the native worker.

Both launch one ExifTool process per path with the same extraction flags and allow 1–36 workers. Rust schedules them through threads; OCaml multiplexes stdout/stderr pipes from child processes. The expensive metadata parser remains ExifTool in both cases. Rust or OCaml execution does not make its metadata a DNG pixel digest.

To select a pool for the portable snapshot:

```
DNG_METADATA_BACKEND="$PWD/rust-scanner/target/release/metadata-extract" \
DNG_METADATA_WORKERS=4 \
```

```
uv run python portable_metadata.py /media/source/foto /media/reference/foto \
  archives/source --batch-size 16
```

The OCaml route uses `DNG_METADATA_BACKEND="$PWD/metadata-extract-oxcaml"`.

Optional exiftool-rs engine

`DNG_METADATA_ENGINE=exiftool-rs` replaces the per-file Perl process with the `exiftool-rs` crate, a pure Rust reimplementation of ExifTool mirroring ExifTool 13.59 tables, running inside the native worker. The build links the crates.io release 0.12.0, pinned exactly, which carries the `-struct` support and numeric-output fixes that dng-compare contributed upstream (listed in `XMP.md`). It is off by default and needs `make build-exiftool-rs`, which rebuilds `metadata-extract` with the `exiftool-rs` cargo feature, builds the standalone OCaml project in `ocaml-exiftool-rs` with dune (OxCaml switch `5.4.0+ox` by default; set `OXCAML_SWITCH=` for the dune on `PATH`), and imports its worker as `metadata-extract-exiftool-rs-oxcaml`. The engine is GPL-3.0-or-later; binaries built with it carry that license. Run it after `make build`: a later `make build-rust` rebuilds `metadata-extract` without the feature, and that binary rejects `--engine exiftool-rs` with an explicit error.

```
DNG_METADATA_ENGINE=exiftool-rs \
DNG_METADATA_BACKEND="$PWD/metadata-extract-exiftool-rs-oxcaml" \
DNG_METADATA_WORKERS=8 \
uv run python portable_metadata.py /media/source/foto /media/reference/foto \
  archives/source --batch-size 16
```

`preserve_years.py --metadata-engine exiftool-rs --metadata-workers N` sets the same variables, records `metadata_engine` and `metadata_engine_version` in the provenance, and archives the engine sources with the tools. Workers accept `--engine exiftool|exiftool-rs` and `--engine-version`; the Perl workers reject the other engine instead of silently substituting it.

Records from this engine are a separate metadata identity. They follow the same `-j -a -G0:1:4 -n -struct -b -u -U` shape and exclusions, with `CopyN` numbering for duplicate tag names in discovery order, nested XMP structures and ExifTool's list rules, but values are still the crate's own: many composites and MakerNotes values keep their print form under `-n`, unknown MakerNotes tags under `-U` are far fewer than Perl's, and `ExifTool:ExifToolVersion` reports the crate version. Every record carries `ExifTool:DngCompareEngine`, and extraction failures are written into the record as `ExifTool:Error` so a failed file remains a failed receipt. Compare archives produced by the same engine only; the Perl flags remain the reference extraction, and `exiftool_version` in the provenance stays the Perl version.

The OCaml binding builds with stock OCaml 5 (`Domain.spawn`) and with OxCaml (`Domain.Safe.spawn` over portable closures), capping workers at the runtime's recommended domain count; the C stub releases the runtime lock so domains run the Rust engine in parallel. Parity with Perl ExifTool on real DNG archives has not been measured here; the crate's published parity claims cover its own corpus. `DNG_METADATA_PREFETCH_BATCHES` defaults to four when a native pool is selected and accepts 1–16. `DNG_METADATA_SHARDS` can run independent extraction/compression shards, with the total native worker budget apportioned across them. A separate index owner receives bounded atomic batches. Shard count must not exceed the worker budget or 16. Only the coordinator writes its checkpoint; workers do not compete for the same SQLite connection.

Year-scoped preservation workflow

`preserve_years.py` wraps byte preservation, full metadata snapshots, XMP extraction, image categorization and final verification for explicit year trees. Supply repeatable labeled source roots and one reference root:

```
uv run python preserve_years.py archives/years \
  --source-root archive-a=/media/archive-a/foto \
  --source-root archive-b=/media/archive-b/foto \
  --reference-root /media/reference/foto \
  --metadata-workers 4 --preservation-workers 2 \
  --metadata-backend rust --preservation-backend rust \
  --database-backend sqlite --metadata-shards 2
```

Only four-digit year directories under those supplied roots become tasks. This is not an automatic whole-volume intake. The source label becomes the archive namespace `LABEL/YEAR` and must remain stable on resume. The matching ledger prefers the same relative path, then a unique case-sensitive filename within the reference year; an ambiguous or missing reference causes full-original fallback. A match never bypasses the subsequent byte verification.

Option	Purpose
`--source-root LABEL=PATH`	Explicit source photo root; repeatable
`--reference-root PATH`	Root containing corresponding reference years
`--start-at LABEL/YEAR`	Skip earlier tasks only after their retained verification passes
`--rate-mib N`	Preservation pacing budget
`--metadata-workers N`	Native ExifTool budget; zero selects serial extraction
`--preservation-workers N`	Byte-preservation pool, 1–36
`--metadata-backend`: `rust`, `oxcaml`	Native extraction implementation
`--preservation-backend`: `python`, `rust`, `oxcaml`	Byte/reconstruction implementation
`--metadata-shards N`	Independent bounded extraction/compression owners
`--database-backend`: `sqlite`, `turso`	Checkpoint engine
`--sequential-pass`	Byte stage followed by metadata
`--unified-pass`	Shared inventory with overlapping byte/metadata work; default

Unified mode lets metadata batches proceed only once the corresponding byte preservation has succeeded. Each stage retains its own checkpoint and database owner. It avoids a second independent inventory in the overlapping phases; it does not avoid reading whatever payloads are needed for actual identity proof.

The workflow records source/reference facts, matching decisions, tool hashes, compiler/extraction settings and available Git state in provenance. Tools are copied into the archive for reproducibility. Keep those artifacts with the payloads. A root-level summary is only the aggregate of its explicitly scoped years, not certification of unrelated lrmobile folders, other volumes or an entire Lightroom library.

A retained archive created by the former hardcoded volume workflow has different scope metadata. Resume it with its archived producing tools or use a new generic output directory; do not silently replace its old scope with newly supplied roots. A same-looking year label does not establish the same source inventory.

Finalization and image categorization

`image_categories.py ARCHIVE` compares DNG raw and auxiliary data after byte preservation completes. Exact whole-file matches can reuse that stronger byte identity; changed copies use the Python image comparison; missing references remain `missing_reference`. `DNG_IMAGE_WORKERS` controls the 1–36 image pool and `DNG_IMAGE_RATE_MIB` controls its pacing estimate. Different encoded images can still require substantial decoding memory.

`finalize_year.py ARCHIVE` validates completed year coverage, recovery records, metadata index integrity, batch checksums, image report coverage, saved originals or delta reconstruction, and source/reference identities. It writes `verification.json`, initially marking validation unfinished before any final acceptance. Inspect its actual issues and constituent reports. Metadata warnings, image differences and recoverable exact originals are separate facts, not one boolean to infer from a log line.

`finalize_preservation.py` is a retained single-archive finalization tool. `export_verified.py` exports only eligible raw-equal Python results for specified roots into the Rust TSV format. `sample_storage_ratio.py` samples archive storage ratios; it is an estimate over that archive, not a predicted universal saving.

```
uv run python finalize_preservation.py archives/source
uv run python export_verified.py reports/full/cache.sqlite3 reports/python-equal.tsv \
  --left-root /media/source/foto --right-root /media/reference/foto
uv run python sample_storage_ratio.py archives/source --output archives/storage-pilot
```

The single-archive finalizer expects the completed byte/metadata/XMP archive layout, not an arbitrary comparison report. The year finalizer additionally expects the year workflow's provenance and tools layout. Both set `safe_to_delete_source` false; validation is archive evidence, not automatic deletion approval. Finalizers take their archive as a positional argument; they are not argparse `--help` interfaces. The export and sampling tools do provide `--help`. Export refuses to overwrite its output. There is no source photo deletion command in this release.

SQLite and optional Turso

The OCaml archive tools and probes open their databases through firstpair-tursolite, one interface over SQLite and Turso whose manual, tursolite, is on the First Pair unix shelf. Archives stay in WAL between runs (`Keep_wal` and `Journal::Existing` there), and closed archives are read with `immutable`, which writes nothing beside the file.

SQLite is the ordinary checkpoint backend. Turso support is pinned to embedded 0.8.3 and uses explicit MVCC transactions during work. The filename extension `.sqlite3` alone does not identify which engine owns a checkpoint; retain its `.engine.json` marker. The wrapper can export a closed checkpoint for legacy reading, but an active archive is not a generic file-copy snapshot.

Migration refuses an uncheckpointed SQLite WAL and keeps a `before-turso` copy. Do not remove ownership markers, edit an active database with another engine, or copy only its main file while the writer has WAL/MVCC state. A retained PID is a liveness guard; inspect a failed owner rather than clearing its state blindly. Ordinary comparison databases are project checkpoints, not Lightroom catalogs.

The Rust `turso-probe` and OCaml `turso_probe_oxcaml.ml`/C bindings are experimental smoke tests for typed values and concurrent writers. The OCaml FFI build requires its matching Turso SDK library and is not part of the ordinary build. Do not confuse a probe's 64 inserted synthetic rows with a photograph comparison or production archive acceptance. `make build-turso` reports the separate supported build path; read its output before using the experiment.

Measure workers without disturbing a real job

Worker count is a measurement, not a core-count rule. HDD seeks, compressed metadata output, ExifTool startup, decoded arrays, database ownership and cache warmth can dominate different phases. Measure a representative, disposable archive scope and compare full extracted values, not just throughput or counts.

The retained benchmark scripts cover native metadata timing, metadata memory, byte verification and the Turso metadata workflow. They write measured inputs, backend choices, typed-value checks and tool hashes to reports. They are research tools with explicit archive/year/filter arguments, not automatic tuning services. Run their `--help` before selecting a scope.

```
uv run python benchmark_metadata_workers.py archives/years --year archive-a/2023
uv run python benchmark_metadata_memory.py archives/years
uv run python benchmark_byte_workers.py archives/years --year archive-a/2023
uv run python benchmark_turso_metadata.py archives/years --year archive-a/2023 \
  --path-contains /selected-album/ --dataset leica
```

The timing benchmark requires 18 selected DNG source/reference pairs and tests worker counts 1, 2, 4, 8, 12, 18, 24 and 36. `--path-contains` explicitly narrows relative paths. The memory benchmark uses the prior timing report's exact input paths. The Turso `--dataset` value labels the result; it does not verify a camera model or select a private camera tree. These scripts still sample live source bytes and create archive-side research outputs, so only run them on a scope you intend to read.

The optional live-job benchmark mode uses SIGSTOP/SIGCONT on a declared owned process tree to isolate a trial. Only opt into that after understanding the selected PID and its descendants. A stopped process can retain locks or an unfinished transaction; stopping an arbitrary Lightroom, database or copy process is unsafe. A Python `finally` attempts resumption during ordinary cleanup, but SIGKILL, a crash or machine shutdown can prevent it. The public benchmark mode requires an explicit pause option and validates the process's ownership and preservation command. Prefer an idle machine and no live PID.

RSS reports sample processes and ExifTool descendants; summed RSS can count shared pages more than once and does not equal unique physical memory. A warmed short metadata trial is not cold multi-terabyte HDD throughput. Report worker count, files, bytes, codec/layout strata, wall time, cache state and whether the measurement includes extraction, compression, indexing and validation.

Recovery after a failure

First determine whether a process is still alive and whether its outputs show actual progress or a terminal error. A stale status file or a command timeout is not proof that the job died. Never start another archive owner because a progress line has been quiet.

Retain the failed report, logs, checkpoint and source version. Rerun the same qualified owner only after its prior process is gone and the source/reference scope still matches. Do not turn an exception into a fabricated equal result. For a decoder failure, use a supported codec/layout or retain the exact original. For a source identity change, determine whether software wrote metadata and start a new qualified comparison rather than weakening the guard.

Temporary `.partial`, `.tmp` or `.recovery-partial` files are incomplete unless their normal owner finished verification and commit. A payload in an archive folder is not sufficient evidence that the whole archive completed. Check the manifest, checkpoint, final summary and restore. For reference-backed storage, check both delta/original payload and the required reference version.

Before deleting any source tree through a separate process, account for all regular files and sidecars, renamed candidates, unique metadata/settings, filesystem attributes, catalog locations and the required independent copies. Perform a verified restore on representative changed and missing-reference cases. A true raw subsumption verdict is useful evidence, but is only one part of that decision.

Boundaries of this release

- Comparison discovery is DNG-specific and name-based; it is not a universal RAW-to-DNG conversion matcher or an automatic renamed-photo detector.
- Selected TIFF page order and tags define the stored-sample scope. Unsupported layouts and codec failures stay unverified.
- Equal stored samples do not mean equal renderings, edits or cloud records.
- No Lightroom catalog, native digest registration, Sync queue or cloud payload is modified.
- ExifTool-visible fields do not cover every private/unparsed byte. Whole-file recovery covers payload bytes when metadata reports cannot explain them.
- Rust and Python caches are filesystem-identity reuse, not an always-fresh cryptographic reread; their identity policies differ deliberately.
- Some low-level tools depend on exclusive ownership by their caller. In particular, Rust report directories are not automatically locked, and Rust metadata batches need an unchanged input pair set and source scope.
- Byte archives depend on their reference copies. Skeletons (Part VII) let a reference take metadata edits; an edit that changes image data or an embedded preview still breaks the records that name it. Full-copy mode and `dngc keep` avoid the dependency at the cost of storage.
- A complete report describes the declared inventory and direction. It does not certify unrelated folders, public cloud convergence or deletion eligibility.

The useful result is an explainable archive decision: what matches, what differs, what remains unverified, and exactly how a retained original can be reconstructed.

Part VI: The archive format

Part II showed what an archive looks like. This part says what it is: the rules a program follows to read one or write one. It is a guide to the specification, docs/dngc-archive-spec.md, which is the normative text and numbers its rules; the section numbers below are its own. The format's name is `dngc-archive-1`.

What the format promises

1. **Exact rebuild.** Every record rebuilds its source file byte for byte, checked against the SHA-256 the record holds.
2. **No overwriting.** When a path's content changes, the new content gets a new record and the old record is kept as an earlier version. A payload file, once written, is never changed.
3. **No silent loss.** A record leaves the archive only by an explicit `drop`, which is logged and can be undone until its trash is emptied.
4. **Reading changes nothing.** A reader creates no file in the archive, not even SQLite's own side files.
5. **Faithfulness.** A file is preserved as it was found: an empty file as empty, a damaged file damaged.

Everything else in the format serves these five.

Terms

- The **source** is the tree an archive preserves, and the **reference** the tree it is preserved against. No tool writes to either.
- A **relative path** is a source file's path below the source root. It is the key of everything the archive holds about that file.
- An **identity** is `[inode, size, mtime_ns, ctime_ns]`, four integers taken when a file was read.
- A **record** is the JSON object that says how to rebuild one content of one source file.
- A **payload** is a file inside the archive that a record names: a delta or a full copy.
- A **version label**, such as `20261029T171631Z-b755b147cbed`, names an earlier content of a path: the source's modification time in UTC and the first 12 hex digits of its SHA-256.

Sections

```
ARCHIVE/
  ARCHIVE.json            every run, migration, drop and keep
  report/                 the comparison: pairs.jsonl, summary.json, cache.sqlite3
  bytes/                  the byte section
    recovery.sqlite3      records, earlier versions, errors, trash, tombstones
    deltas/               deltas/<relative path>.vcdiff
    originals/            originals/<relative path>
    versions/<label>/     payloads of contents that replaced an earlier one
  metadata/               batches/*.jsonl.gz and differences.sqlite3
  skeletons/              skeletons.sqlite3 (Part VII)
  trash/<drop id>/        payloads of dropped records
```

Every section is optional. An archive with only `report/` is a comparison; with `bytes/` it can rebuild files; with `metadata/` it can be queried; with `skeletons/` its reference may be edited. Records name their payloads relative to the byte section, so the whole directory can be moved or copied.

A program given a path finds the byte section by the rule in Part II, *The byte-section rule* (specification section 3). All implementations use that one rule, so an archive written before the `bytes/` folder existed is still read where it lies.

The archive's own record

`ARCHIVE.json` is one JSON object with `schema` (`"dngc-archive-1"`) and four lists: `runs`, `migrations`, `drops` and `keeps`. Each entry says when it happened, what it was given and how it ended. A run records both roots, the pair file, the implementation and the exit status of each step. The file is rewritten whole, to a temporary name and then renamed, and a writer keeps any member it does not know.

The byte section's database

`bytes/recovery.sqlite3` is an SQLite database with three tables, and two more after the first drop:

```
CREATE TABLE files    (path TEXT PRIMARY KEY, record TEXT NOT NULL);
CREATE TABLE versions (path TEXT NOT NULL, version TEXT NOT NULL, record TEXT NOT NULL,
                       PRIMARY KEY (path, version));
CREATE TABLE errors   (path TEXT PRIMARY KEY, error TEXT NOT NULL);
CREATE TABLE trash    (drop_id TEXT NOT NULL, state TEXT NOT NULL,
```

```
                      path TEXT NOT NULL, version TEXT NOT NULL, record TEXT NOT NULL);
CREATE TABLE dropped  (path TEXT PRIMARY KEY, drop_id TEXT NOT NULL);
```

`files` holds the current content of each path, `versions` its earlier contents by label, `errors` the files that could not be preserved. A record's text is JSON exactly as Python's `json.dumps` writes it, with `", "` and `": "` separators and keys in a fixed order. That is a rule, not a habit: three implementations write records, and the tests compare what they write.

Because the records are JSON in SQLite, an archive can be asked questions directly:

```
sqlite3 'file:/Volumes/Archive/dngc/2024/bytes/recovery.sqlite3?immutable=1' \
  "SELECT path FROM files WHERE json_extract(record, '$.delta') IS NOT NULL LIMIT 10"
```

Records and their three kinds

Part II, *What a record holds*, shows a record. Its kind is decided in this order:

Kind	How it is recognized	The source is
delta	`delta` is a non-empty string	the reference with the delta applied
full	otherwise, `original` is a non-empty string	that payload
identical	neither	the reference's bytes

A writer adds a record only after proving it: a delta is decoded again and the result's SHA-256 compared with `source_sha256` before the record exists. `reconstruction_verified` is therefore true in every record.

Deltas are VCDIFF (RFC 3284), encoded against the whole reference file. dng-compare uses `xdelta3`, and any VCDIFF decoder that takes the reference as its source rebuilds the file. A payload is written to a temporary name, synced and renamed into place.

Restoring a content

To restore a relative path, or an earlier content of it by label, a reader:

1. refuses if the output exists, and writes to `OUTPUT.recovery-partial`;
2. for a delta, checks that the reference's SHA-256 is `reference_sha256` and decodes the delta against it; for a full record, reads the payload; for an identical record, reads the reference;
3. checks that the SHA-256 of what it wrote is `source_sha256`, and on a mismatch removes the partial file and fails;
4. syncs, links the partial file to the output and removes the partial name;
5. restores the source's mode, times, flags and extended attributes from `source_attrs`.

Step 3 is what makes every other shortcut safe. Whatever a reader did to find its reference, a wrong byte cannot get past the final check.

A reference is a content, not a path

A record gives its reference twice: as a path (`reference`, and `reference_relative_path` below the reference root) and as a SHA-256. The format treats the SHA-256 as what the record depends on and the path as where that content was last seen. A reader looks for the content in this order:

1. at the recorded path, or below `--reference-root` when the reference tree has moved;
2. in any other file that holds the same content. A reader can be given phindex indexes for this and finds such a file by an indexed lookup of its SHA-256, checking the file's digest before using it;
3. by rebuilding it from its skeleton and an edited version of the file (Part VII), found at the recorded path or, with an index, among the files that share its image data.

The front ends do steps 1 and 3 at the recorded path. Step 2, and step 3 for a file that was moved as well as edited, need an index: `--index FILE` on `dngc restore`, `dngc verify --restore-sample` and `dngc keep` in both front ends, `Reader::with_indexes` in the Rust library, `~indexes` on `Preserve.restore` in the OCaml library and `content_index.py` in the Python reference. The same three commands take `--reference-root DIR` for step 1. An index is an aid only: a stale or wrong entry costs time and is then rejected by the SHA-256 checks. That time is bounded: for one record a reader by default digests at most 8 listed files in step 2 and tries at most 8 edited files in step 3, reading four times as many index rows per lookup, and then fails as it would without an index. `--candidates N` (`Reader::with_candidates(n)` in Rust, `~candidates` in OCaml) sets another limit; 0 turns the index steps off.

The comparison and metadata sections

`report/pairs.jsonl` has one JSON object per pair, with sorted keys and no spaces: both paths, both files' stamps, the status of the raw image and of the auxiliary images (`equal`, `different`, `unverified` or `not_compared`, with the method that decided it), and the metadata differences. `cache.sqlite3` lets a repeated run reuse a result while both files' stamps are unchanged.

`metadata/batches/*.jsonl.gz` holds each file's full metadata record on both sides, and `differences.sqlite3` one row per changed tag, which `dngc query` reads. The index can always be rebuilt from the batches (`dngc index`). Part V, *Full metadata snapshots and queries*, describes its tables; the specification's section 7 gives their exact definitions and the columns a resumed run relies on.

Dropping, the trash and tombstones

`dngc drop --yes` moves the selected records to the `trash` table and their payloads to `trash/<drop id>/`; `--force` deletes them instead. Either way each path gets a **tombstone** in `dropped`. A tombstone does three things: a later run does not preserve that path again, `dngc index` removes its metadata rows, and `restore` of it fails, naming the drop. `trash restore` puts a drop back, `trash empty` deletes its payloads for good and keeps the tombstones, and `trash release` removes the tombstones so that later runs preserve those paths again.

`dngc keep --yes` goes the other way: it turns identical and delta records into full copies inside the archive, after which they no longer depend on the reference at all.

Locks and reading

Each section has one lock file, and a writer holds an exclusive `flock` on it for as long as it writes. `migrate`, `drop`, `trash` and `keep` take every lock without waiting and refuse if one is held.

A reader opens a database with SQLite's `immutable=1` when no writer holds its lock and it has no pending write-ahead log, and otherwise reads a private copy. That is how promise 4 is kept: an archive on a backup volume, or on read-only media, is read without a single file appearing beside it.

Conformance

A format with three implementations needs a judge that is none of them. `conformance/validate_archive.py` checks an archive against the specification and names the rule each problem breaks. It is written from the specification alone and imports nothing from dng-compare:

```
uv run python conformance/validate_archive.py /Volumes/Archive/dngc/2024 --sample 200
```

`--rebuild` rebuilds every record from its reference, and `--sample N` only N of them.

`conformance/corpus/` holds twenty small archives, some valid and some broken on purpose, each with the list of rules the validator must report for it: a missing payload, a wrong hash, a record not in `json.dumps` form, a tombstone beside a live record, a corrupt skeleton and others. Each implementation is then tested twice over:

- **reading:** it must restore every record of every valid case byte for byte, and must refuse the broken ones;
- **writing:** what it writes from a case's trees must pass the validator.

`make conformance` runs the Python, Rust and OCaml suites in turn, the OCaml one under OxCaml; `make conformance OXCAML_SWITCH=default` runs it under stock OCaml. The same tests check the two libraries that other programs build on: `dng-archive` in OCaml (`ocaml-archive/`) and `dng-archive` in Rust (`rust-archive/`).

The suite has already earned its keep. Writing the validator from the text showed that the specification said every pair result carries `stamps`, while all three implementations leave them out for a pair that failed before its files were read. The specification was wrong and was corrected.

Part VII: Skeletons

The problem: a reference that is still in use

A delta record and an identical record both say "the source is that reference file, perhaps with this small change". They are cheap because the reference holds the bytes. They are also fragile for the same reason: they need the reference's exact bytes, checked by SHA-256.

A reference is usually not a dead copy. It is the working library: the tree a photo editor has open. When Lightroom writes a rating, a crop or an exposure adjustment into a DNG, or Photo Mechanic writes a caption into a JPEG, the file's bytes change, its SHA-256 changes, and every record that names it stops restoring. Nothing is damaged, and yet the archive has lost a file.

The usual answers are both poor. Freezing the reference means not using the library. Keeping full copies means storing every file twice. Skeletons are a third answer: keep, per reference file, only what an edit destroys.

What an edit actually changes

A raw file is a small amount of structure wrapped around a few large blocks. In a DNG the structure is the TIFF header, the image file directories with their tags, and the XMP packet; the blocks are the raw image's strips or tiles, the embedded previews, the maker notes and the camera profiles. An editor's settings, including every develop adjustment, live in the XMP packet. Writing them rewrites the packet and the directories, and often moves the blocks to other offsets because the packet grew. It does not change a byte inside a block.

This was measured, not assumed. One archive held 498,128 deltas between an older copy of a library and the same library after years of use in Lightroom Classic and Photo Mechanic: real edits, with ratings, keywords, crops and develop settings (exposure, blacks, shadows and tone curves differed in about 13,000 pairs). Of those deltas 497,790 were smaller than 4 KB. In all but a few hundred files, everything except a few kilobytes of the old file was still present, somewhere, in the new one.

Blocks and the skeleton

A file is split by its own structure into **blocks** and the **skeleton**:

- a **block** is a large run of bytes that an edit carries over unchanged;
- the **skeleton** is every byte that is not in a block, in file order.

Only the skeleton is stored. For each block the archive keeps where it was (offset and length), what it is (a role such as `0/330.0/tiles`, the tiles of the first sub-image), and a **mark**: a hash of the block's first and last 4,096 bytes.

For a TIFF-based file, which covers DNG and most camera raws, the blocks are:

- the strips or tiles of each image, merged into one block when they lie within 3 bytes of each other, since encoders pad them;
- each embedded JPEG stream;
- any other tag value of 512 bytes or more, except the three that editors rewrite: XMP, IPTC and Photoshop resources;
- any run of 64 KB or more that no tag points at. Some cameras keep data there that only their own maker notes reach; in one maker's files it was 620 KB per picture.

For a JPEG the blocks are each segment of 512 bytes or more other than Exif and XMP (APP1), IPTC (APP13) and comments, and everything from the start of scan to the end of the file.

A file of any other format up to 1 MiB, such as an XMP sidecar, is its own skeleton: it is simply stored whole. Larger files of other formats, video above all, have no skeleton.

The specification's section 13.1 gives these rules exactly, down to the order in which overlapping candidates are dropped, because three implementations and the validator must cut every file in the same places.

Rebuilding a file from its edited version

Given the skeleton of a file as it was, and the file as it is now:

1. walk the edited file's structure and compute the mark of each of its blocks;
2. write the old file in order: skeleton bytes, then for each block the bytes of the edited file's block with the same length and mark, then more skeleton bytes;
3. compare the SHA-256 of what was written with the old file's SHA-256.

If a block of the old file is not in the edited one, step 2 fails. If a block has the same ends but differs in the middle, the mark matches and step 3 fails. Either way nothing wrong is produced. The mark is a cheap way to find a block; the SHA-256 is what proves the result.

Writing a skeleton reads only the file's structure, the bytes between its blocks and the two ends of each block: a few small reads per file, never the image data. That is why skeletons for a whole library can be written in hours, on disks where reading every byte takes days.

The skeleton section

`ARCHIVE/skeletons/skeletons.sqlite3` has one table with one row per content, keyed by the content's SHA-256:

```
CREATE TABLE skeletons(sha256 TEXT PRIMARY KEY, record TEXT NOT NULL, skeleton BLOB NOT NULL);
```

`skeleton` is the skeleton's bytes, compressed with zstd. `record` describes it:

```
{"sha256": "705335…", "size": 10435448, "format": "tiff",
 "path": "/Volumes/Backup/2011/April/IMG_0332.dng",
 "identity": [183, 10435448, 1771560403805486318, 1771560403805486318],
 "blocks": [[2048, 65000, "9c1d…", "0/tag50740"],
            [131072, 10200000, "41aa…", "0/330.0/tiles"],
            [10331072, 100400, "e07b…", "0/330.1/strips"]],
 "skeleton_bytes": 17858, "skeleton_sha256": "b2f0…", "skeleton_format": "zstd"}
```

A skeleton belongs to a content, not to a path. Two records that name the same reference content share one skeleton, and a skeleton still serves after its file is renamed.

Writing skeletons

```
dngc skeleton build /Volumes/Archive/dngc/2024 --workers 4
dngc skeleton info  /Volumes/Archive/dngc/2024
```

`build` writes a skeleton for every reference file the archive's records name. It must run while those files still hold the contents the records saw. It does not read a whole file to make sure, so it takes a content's SHA-256 from the record only while the file's size and modification time equal the record's `reference_identity`; a file that has already changed gets no skeleton and is listed in `skeletons/skipped.tsv` with the reason. A content that already has a skeleton is skipped, so an interrupted build continues where it stopped, and a build after a later `dngc run` adds only the new references.

The native `dngc` builds skeletons itself. `dngc.py` runs the Rust tool, `dngc-skeleton`, which is also the fastest of the three and has one more verb:

```
dngc-skeleton check /Volumes/Archive/dngc/2024 --sample 1000
```

`check` rebuilds a random sample of contents from their skeletons and the files now at their paths, reading those files in full, and compares each with its SHA-256. Run it after a build, and again whenever you want to know that the reference, however much it has been edited since, still carries everything the archive needs.

Restoring through a skeleton

Nothing changes in how an archive is used. `restore`, `keep` and `verify --restore-sample` do the extra step on their own:

- for a **delta** record, when the reference's SHA-256 is no longer `reference_sha256`;
- for an **identical** record, when the reference's size or modification time is no longer what the record saw.

The reader then looks up the skeleton of the content it needs, rebuilds that content from the edited file into a temporary file, checks its SHA-256, uses it as the reference and removes it. An archive without a skeleton section, or without a skeleton for that content, fails exactly as it did before, with the same message.

`dngc verify` reports how many skeletons an archive holds and checks their database.

What skeletons do not cover

A skeleton can only return what the edited file still contains.

- **A regenerated preview.** Lightroom's *Update DNG Preview & Metadata* renders a new embedded preview; the old one is gone. Saving metadata alone does not do this.
- **Changed image data:** a conversion to lossy DNG, a re-encode, a pixel edit saved over the original.
- **A truncated or damaged file.**
- **Formats without a walker** above 1 MiB: video, PSD and containers such as CR3 and HEIC.

In each case rebuilding fails and says which block is missing. To keep such a content, preserve it from a copy that still has it before that copy is updated: a second copy of the reference, synchronized only after `dngc run` has archived what changed, turns every such loss into an ordinary new version.

Skeletons also do not replace `dngc keep` for files you mean to stop depending on a reference for.

Measured

On pairs taken at random from the archive described above, old copy against edited copy:

Sample	Rebuilt exactly from the edited file	Skeleton stored, average
1,494 DNGs	1,494	6.5 KB
400 JPEGs	399	9.7 KB

The one JPEG that failed was truncated in the old copy: 131,072 bytes against 11 MB.

The archive's 298 pairs with deltas of 4 KB or more were tried too, with an early version of the walkers. In 242 of them the image data itself differed between the copies, and rebuilding failed as it should. In 4 the marks matched and the final SHA-256 check caught the difference.

Writing skeletons for the first 20,000 reference files of that archive, from hard disks, took 611 seconds with four workers, used 367 MB of memory at its peak and stored 164 MB: 4.5 KB per DNG and 13.9 KB per JPEG, whose Exif segment with its thumbnail is kept whole. The files skipped were 69 videos.

Against that, the same files as full copies would have taken the size of the library again.